

Highlights

LEAP: A Literate-Programming Architecture for Reconstructing Traceable KPI Knowledge from Enterprise Codebases

Meshaal Obaid Mouawad, Bryan A. Jones

- LEAP links executable KPI evidence, formulas, business context, and review state.
- Literate-programming principles guide a source-linked KPI knowledge record.
- A worked example connects source annotations to a generated KPI dossier.
- Verification separates discovery, prepared-record scenarios, and MATE checks.
- Optional AI-assisted authoring remains separate from source-derived evidence.

LEAP: A Literate-Programming Architecture for Reconstructing Traceable KPI Knowledge from Enterprise Codebases

Meshaal Obaid Mouawad^{a,*} (Researcher), Bryan A. Jones^a (Professor)

^aDepartment of Electrical and Computer Engineering, Mississippi State University, Box 9571, Mississippi State, 39762, MS, USA

ARTICLE INFO

Keywords:

key performance indicators
program comprehension
software documentation
literate programming
static analysis
knowledge engineering
traceability

ABSTRACT

Enterprise key performance indicators (KPIs) are implemented in source code while their business definitions, mathematical descriptions, and review records are often maintained separately. This separation makes discrepancies difficult to trace across technical and business representations. We present LEAP (Literate Programming for Automated KPI Extraction), an architecture for reconstructing a source-linked KPI knowledge record and publishing it through an *Interactive Bluebook*. The method applies literate-programming coupling to heterogeneous implementation artifacts. It combines annotation-based and structural candidate selection, preparation of declared and supported source expressions, mathematical annotation, captured source context, and bounded review signals. Declared meaning, executable evidence, and optional AI-assisted explanation remain distinguishable rather than being treated as interchangeable authorities.

Evaluation uses a frozen research artifact, a worked transformation, archived executions, and a historically traced prepared-record acceptance suite. The archived snapshot verification reports 19 passing regression checks and 38/38 prepared-record scenarios passing. The latter exercise downstream detail and governance behavior, not repository discovery or formula equality. Four archived runs each produced 62 KPI records from the same 65 selected source files and 527 analyzed lines in 0.93–1.16 seconds; a broader run generated 148 dossiers in 2.73 seconds. These results establish bounded implementation behavior and execution characteristics, not independently measured extraction accuracy. The contribution is the KPI-specific source-to-record method and its evidence responsibilities, including preservation of disagreement through publication; it is not a new general formula-extraction or provenance algorithm.

1. Introduction

Modern enterprises depend on key performance indicators (KPIs) for operational control and strategic decision-making (Parmenter, 2015; Kaplan and Norton, 1996). Yet a KPI rarely exists as one coherent artifact. Its intended meaning may be maintained in a catalog, its calculation in source code, its presentation in a dashboard, and its ownership in a governance register. The executable result is produced by software, while the explanation used by analysts and decision makers is maintained elsewhere.

This separation creates a knowledge discontinuity between technical implementation and business interpretation. A code change can alter a KPI while its documentation remains unchanged; a definition can likewise change without a corresponding implementation update. We use *KPI semantic drift* to describe divergence among intended meaning, documented meaning, and executable logic. The engineering problem is therefore not only to find KPI-like calculations, but to recover an evidence chain that relates source regions, formulas, definitions, ownership, review state, and published documentation.

This paper presents LEAP (Literate Programming for Automated KPI Extraction), a research artifact for reconstructing reviewable KPI knowledge from heterogeneous enterprise codebases. Here, *governed* denotes configured

ownership, review signals, and rule metadata within the artifact; it does not establish organizational approval or external certification. The work follows a four-level hierarchy: literate programming provides the conceptual foundation; enterprise KPI knowledge engineering defines the problem; deterministic analysis with optional AI-assisted authoring defines the computational approach; and the *Interactive Bluebook* is one published output. The Bluebook is therefore not the system itself, but a navigable view over the recovered knowledge records.

One implementation principle separates evidence construction from optional authoring:

Rule-based analysis selects and represents source evidence under explicit conditions. Optional AI-assisted authoring may explain a prepared record, but it does not assign its review status.

The literate-programming contribution is therefore not simply documentation generation from code. Classical literate programming keeps executable representation and human explanation connected. LEAP generalizes that coupling to a heterogeneous enterprise setting by preserving code regions, mathematical representation, business meaning, provenance, ownership, and review signals as linked fields of one knowledge record. The *Interactive Bluebook* is a publication surface over that record rather than a detached narrative copy.

LEAP uses fixed rules for discovery, candidate selection, preparation of declared formulas and supported executable

*Corresponding author

 mmouawad@ece.msstate.edu (M.O. Mouawad);

jones@ece.msstate.edu (B.A. Jones)

ORCID(s): 0000-0002-1152-8324 (M.O. Mouawad)

expressions, mathematical annotation, source capture, and bounded review checks. Selection is conditional on file and candidate order: the frozen traversal is unsorted, and duplicate-name retention can therefore vary with traversal order. Optional AI-assisted authoring is disabled for the reported verification and is not required for record construction or publication.

1.1. Research questions

The evaluation is organized around three technical research questions:

- RQ1** How can literate-programming principles be operationalized in an architecture for constructing traceable KPI knowledge records from heterogeneous source evidence?
- RQ2** What source-to-record, mathematical-annotation, publication, and review-routing behaviors are demonstrated by the artifact's verification layers?
- RQ3** What execution characteristics are observed in archived runs at the curated and broader repository scopes?

This paper studies the architecture, record-construction method, and artifact behavior. Governance effectiveness, AI-assisted authoring quality, and human or organizational outcomes are reserved for separate studies. Independent extraction and formula-fidelity evaluation is not inferred from software tests.

1.2. Contributions

The paper makes three technical contributions, supported by a reproducibility package:

1. **A literate KPI knowledge architecture.** We define a source-to-knowledge pipeline that connects executable evidence, mathematical representation, business context, ownership, review state, and provenance.
2. **A source-to-record construction method.** We specify the implemented candidate-promotion and selection procedure, the Mathematical Annotation and Tagging Engine (MATE), and the provenance retained by the published record.
3. **Traceable mathematical representation and bounded review signals.** We connect mathematical presentation, source context, and bounded review checks, including a small set of rule-mapped scenarios and ISO 22400 templates; exhaustive governance evaluation is outside this paper's scope.

An accompanying reproducibility package binds tests, fixtures, generated outputs, and archived executions to a claim register. It supports inspection of these contributions rather than constituting an additional algorithmic invention.

1.3. Artifact availability

The review package contains a frozen software snapshot, evidence archive, claim matrix, tests, controlled fixtures, generated Bluebook pages, and the manuscript source. The frozen snapshot derives from commit `f3b7b1d7af61`; the accompanying software manifest records the full identity. This review version does not assert a public archival identifier.

2. Background and Related Work

2.1. Literate programming and executable documentation

Knuth's literate programming paradigm treats programs as explanations intended for human readers as well as instructions for machines (Knuth, 1984). Elucidative Programming later showed that source and explanation can remain in separate artifacts while preserving explicit, navigable relations between them (Nørmark, 2000). Computational notebooks further lowered the barrier to combining executable content and narrative (Rule et al., 2018; Kluyver et al., 2016). However, enterprise systems differ from notebooks in three important respects: they are distributed across repositories, implemented in multiple languages, and changed by teams with different roles and incentives. Manually maintained narrative therefore tends to lag behind implementation.

A related but opposite transformation direction is represented by Drasil: domain knowledge is encoded explicitly and then used to generate mutually consistent software and documentation artifacts, improving traceability and change management (Carette et al., 2023). LEAP instead starts from existing heterogeneous implementations and reconstructs a reviewable KPI record around them. This distinction matters because the source code already exists and may disagree with comments, formulas, or business metadata.

LEAP extends the literate-programming principle through this reconstruction direction. Rather than treating prose as a detached derivative of code, it maintains linked record components for executable source, formula presentation, business interpretation, provenance, ownership, review state, and published explanation. Following the cross-artifact coupling already demonstrated by elucidative programming, LEAP specializes that relationship to a KPI record with explicit evidence origins and review responsibilities; separating code and explanation is not itself claimed as novel. The approach does not replace human review; it preserves the connection between implementation and explanation while making that connection inspectable and regenerable. Large-scale notebook studies also show that combining code and narrative does not by itself guarantee reproducibility, reinforcing the need for explicit provenance and validation state (Pimentel et al., 2019).

2.2. Mathematical documentation from source code

Source-to-mathematics documentation predates LEAP. RbG extracts mathematical formulas and decision tables through static analysis and uses source annotations to structure generated OpenDocument or LaTeX documentation

(Moser et al., 2015). Moseler et al. identify formula-code patterns in Java and reconstruct selected sum and product computations in mathematical notation (Moseler et al., 2021). Their contribution concerns formula-code structure and occurrence, whereas LEAP's unit of analysis is a KPI record assembled from potentially inconsistent implementation and declaration evidence.

MATE should therefore be understood as a component of this record-construction method, not as the invention of formula extraction or mathematical redocumentation. Its role is to prepare supported source expressions and declared formulas, expose mathematical and token views, and retain the source context and bounded disagreement signals needed to interpret those views. LEAP does not reproduce the formula-recovery evaluations of these predecessors or claim comparative accuracy.

2.3. Program comprehension and code summarization

Program comprehension research has examined static analysis, code naturalness, learned representations, and natural-language summaries (Allamanis et al., 2018). Empirical work on software documentation shows that creation and maintenance problems are persistent in practice (Aghajani et al., 2019). Recent work has also evaluated LLMs for documentation-to-code traceability and found that model-assisted link recovery can be useful while still producing false links and relationship errors that motivate human review (Alor et al., 2026). For enterprise KPI documentation, these limitations matter because stakeholders need not only a fluent description but also evidence that connects each statement to executable logic.

LEAP therefore adopts a hybrid architecture. Static and rule-based methods establish the structural and mathematical representation; optional AI-assisted authoring receives the prepared record, source evidence, and context and may produce business-facing explanation. This reduces reliance on unconstrained generation and keeps source anchors, token links, and origin categories explicit for the record fields that the artifact can support. AI-assisted authoring is an architectural extension point, not part of the validation authority evaluated in this paper.

2.4. KPI modeling, assessment, and evolution

The Balanced Scorecard (Kaplan and Norton, 1996) and Goal–Question–Metric (van Solingen et al., 2002) organize measures, while management-oriented KPI literature discusses their selection and use (Parmenter, 2015; Eckerson, 2011; Marr, 2012). Technical KPI approaches already integrate definition, calculation, visualization, and traceability. MIKADO builds executable assessments and dashboards from KPI and domain models (De Sanctis et al., 2022). Its ethical-risk extension links expert-assessed risks to KPI inputs and exposes warnings in generated results (De Sanctis et al., 2025). Review-relevant warnings in KPI presentation are therefore not unique to LEAP; the distinction here is their connection to reconstructed source and declared-formula evidence.

AFFECTK models KPI evolution, consistency, and traceability through configurable evolution patterns (Domínguez et al., 2025). Its prototype executes primary model transformations, including creating or deleting KPIs and updating calculation rules. Consequent propagation actions are recorded as pending optional or compulsory decisions rather than all being applied automatically. This is model-evolution management, not reconstruction of a KPI's implementation from source. LEAP begins with existing heterogeneous artifacts and retains disagreement among their representations; it does not claim AFFECTK's longitudinal evolution-management capabilities.

2.5. Traceability, software knowledge, and boundary objects

Recovery of links between documentation and source code is an established software-engineering problem (Marcus et al., 2005). Putrycz and Kark further connected business rules recovered from legacy source code to existing documentation for business-facing use (Putrycz and Kark, 2008), making code-to-business knowledge reconstruction an important direct predecessor rather than a new problem introduced by LEAP. Traceability research also emphasizes that these links must be maintained as systems evolve (Mäder and Gotel, 2012). Industrial experience shows that missing or decayed traceability can directly obstruct software-quality and maintenance work (Fucci et al., 2022). Software knowledge graphs provide another adjacent approach for organizing code, architecture, API, and other engineering knowledge; a recent systematic review documents broad use across software engineering while also showing that knowledge-graph designs vary substantially by task and modeled content (Wang et al., 2023).

The *Interactive Bluebook* is designed around the boundary-object principle of retaining a shared referent across different communities (Star and Griesemer, 1989): developers inspect functions, variables, and provenance; analysts inspect business meaning, units, and assumptions; governance staff inspect ownership, evidence, exceptions, and review state. The contribution in this paper is not a general knowledge graph or a generic trace-link predictor. It is the construction of a KPI-specific record whose mathematical and human-facing fields remain linked to source evidence.

2.6. Contemporary source-bound knowledge representations

Recent source-bound documentation systems further narrow the comparison. CODENS incrementally turns code changes and pull-request history into structured, queryable documentation using a typed software knowledge graph and semantic extraction (Kelious et al., 2026). Gao's Executable Code Knowledge (ECK) preprint defines source-bound knowledge units that carry semantics, executable evidence, provenance, validation state, and query interfaces for AI coding agents (Gao, 2026). These systems reinforce the premise that useful software knowledge should remain

bound to source and validation evidence rather than exist only as free-standing generated text.

The distinction for LEAP is therefore not merely “agents versus KPIs.” CODENS emphasizes change history and queryable repository knowledge; ECK emphasizes authored or maintained code-bound units, evidence execution, and freshness. LEAP reconstructs KPI-specific records from existing heterogeneous artifacts, separates declared mathematical meaning from executable evidence, prepares business/developer formula views, and applies bounded review-state rules before dossier publication. The novelty claim in this paper is limited to that reconstruction method and evidence responsibility model, not to source-linked software knowledge in general.

2.7. Position relative to adjacent research

The relevant research traditions are complementary rather than competitors. Literate and elucidative programming establish code–explanation relations (Knuth, 1984; Nørmark, 2000); traceability and reverse-engineering work connects implementation artifacts to documentation and business rules (Marcus et al., 2005; Putrycz and Kark, 2008; Mäder and Gotel, 2012); mathematical-documentation approaches extract or reconstruct supported formula forms (Moser et al., 2015; Moseler et al., 2021); knowledge-generation approaches such as Drasil connect formalized domain knowledge to multiple generated artifacts (Carette et al., 2023); KPI frameworks such as MIKADO and AFFECTK integrate modeling, calculation, evolution, and traceability (De Sanctis et al., 2022; Domínguez et al., 2025); and recent knowledge-graph and LLM work constructs source-connected documentation and trace links (Wang et al., 2023; Alor et al., 2026; Kelious et al., 2026). LEAP combines selected ideas from these traditions around a narrower transformation: reconstruction of a KPI record from existing heterogeneous implementation evidence. We therefore avoid assigning binary capability scores to entire research traditions and make only this bounded integration claim.

LEAP’s theoretical connection to literate programming remains central. Classical literate programming organizes narrative and code for human comprehension (Knuth, 1984). LEAP extends this idea by recovering and maintaining a structured chain from executable implementation to mathematical meaning, business interpretation, assumptions, provenance, review state, and published documentation. In this sense, LEAP operationalizes literate programming as a form of *executable knowledge engineering* for heterogeneous enterprise repositories.

2.8. Comparison with representative industrial platforms

LEAP overlaps with adjacent industrial categories rather than serving as a direct substitute for a single platform. Palantir Foundry documents data integration and ontology-centered operations (Palantir Technologies, 2026); Collibra and Alation document catalog, governance, and lineage capabilities (Collibra, 2026; Alation, 2026); SAP LeanIX

documents enterprise-architecture management (SAP, 2026); and Sourcegraph documents source-code navigation and code intelligence (Sourcegraph, 2026). LEAP’s narrower contribution begins at executable source evidence and constructs a KPI record that joins formula structure, provenance, ownership, deterministic review signals, and a published dossier. This positioning is conceptual; the paper does not claim a comparative product-performance evaluation.

2.9. Research gap

Prior approaches already connect code with explanations, recover business rules, generate software from domain knowledge, reconstruct selected mathematical formulas, and manage modeled KPI evolution (Nørmark, 2000; Putrycz and Kark, 2008; Moser et al., 2015; Moseler et al., 2021; Carette et al., 2023; De Sanctis et al., 2022; Domínguez et al., 2025). Recent systems also construct source-connected documentation and executable knowledge (Kelious et al., 2026; Gao, 2026). Neither source linkage nor mathematical presentation alone defines LEAP’s contribution.

The problem addressed here is the construction of a KPI-specific record from existing heterogeneous artifacts when a declaration, source calculation, business description, and review state may disagree. LEAP specifies how candidate selection, formula preparation, evidence-origin distinctions, and bounded review signals produce an inspectable dossier without silently treating one representation as proof of another. The contribution lies in that operational connection and its implementation, rather than a claim of firstness for the individual components or demonstrated superiority over the preceding systems.

3. LEAP Architecture and Method

3.1. Design principles and architectural rationale

The design addresses a specific form of documentary separation: a source expression, its mathematical description, and its business interpretation may change independently. Literate programming motivates keeping explanation connected to executable representation (Knuth, 1984). For existing enterprise repositories, LEAP applies that principle through reconstruction rather than requiring developers to rewrite all software as literate source. The resulting record is a navigable connection among artifacts, not a replacement for the original implementation.

Table 1 maps this rationale to the implemented mechanisms in Figure 1. It is an explicit account of the artifact’s design choices, not a claim that a prospective design-science or independent practitioner-validation study was conducted. In particular, fixed-rule execution permits inspection of how a record was constructed; reproducibility also depends on input and traversal order. Neither property establishes the correctness of a formula or business interpretation.

3.2. Architectural overview

Source artifacts pass through discovery, language routing, candidate selection, formula preparation, provenance capture, and review evaluation. The resulting records are published

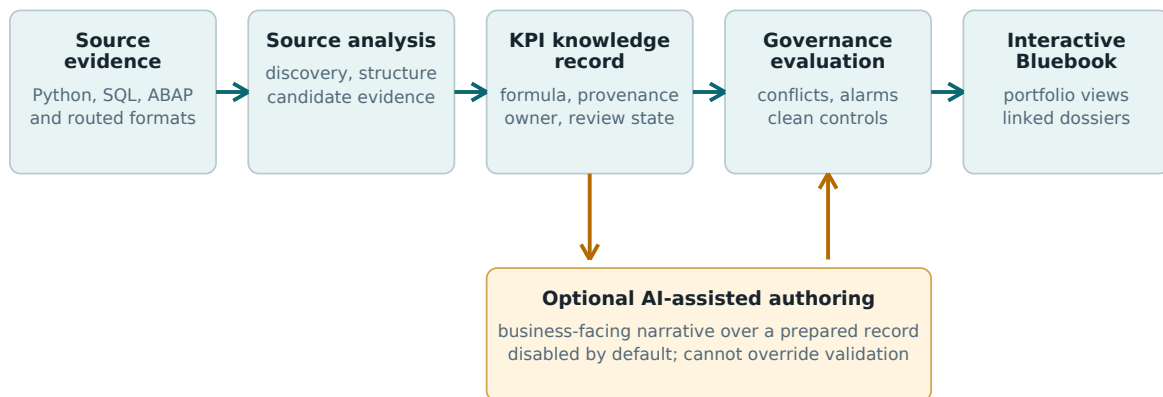
Table 1

Design rationale linking the KPI knowledge problem to the architecture.

Principle	Problem addressed	Architectural mechanism
Literate coupling	Formula, explanation, and implementation become detached.	One dossier exposes the formula and business context alongside the recorded source region.
Evidence origin	An annotation or generated explanation can be mistaken for executable behavior.	Keep source text, declared metadata, rendered formula, and review signals distinguishable.
Inspectable reconstruction	A document can outlive the source state it describes.	Re-run source analysis and record preparation; bind the evaluated package to a software manifest.
Explicit validation responsibility	Fluent narrative can obscure an unresolved source conflict.	Rule and review signals are assigned outside optional AI-assisted authoring.
Role-preserving views	Stakeholders need different representations of the same calculation.	Business and developer formula perspectives share a dossier and source context.
Bounded automation	Unsupported candidates can appear to be validated metrics.	Promotion gates and review records retain reasons for withholding or reviewing a dossier.

LEAP source-to-knowledge architecture

Source evidence, mathematical presentation, and review information remain connected.



Source-linked records with separate analysis and authoring responsibilities

Figure 1: LEAP source-to-knowledge architecture. Source analysis constructs a record; formula, provenance, and review information remain linked at publication. The optional-authoring connection represents narrative data flow over a prepared record; review and validation status remain assigned by the non-generative rule/review path.

through the *Interactive Bluebook*. These stages are implemented with parsing, patterns, and rules. Optional AI-assisted authoring may express a prepared record for human readers; it does not determine the source evidence or assign validation status.

The distinction is operational, not a comparison claiming superiority over an AI-based extractor. AI-assisted authoring was disabled for the reported verification. Table 2 defines the responsibilities relevant to reproducing this configuration.

3.3. Hybrid candidate extraction and selection

The implemented hybrid method combines explicit declarations with structural and naming patterns. It is not specified by a learned weighted threshold. The extractor inventories the requested directory, excludes configured paths and non-source artifacts, and routes eligible text files by extension. A candidate stores a normalized name, a detection kind, a file-line anchor, and local `code_context`. Common comment and Python routes use a window of up to ten preceding and twenty following lines; some SQL and .NET paths expand

Table 2
Evidence construction and optional authoring responsibilities.

Responsibility	Implemented or evaluated role
Source analysis	Discover candidates, preserve local context, and select records through implemented rules.
Mathematical presentation	Prepare supported formula forms and business/developer annotations.
Review evaluation	Attach configured ownership, conflict, technical, and rule-mapped signals.
Publication	Generate dossiers and portfolio views from the prepared records.
Optional authoring	May produce business-facing explanation over a record; disabled in this evaluation.

that context. This is a recorded source region, not necessarily a complete interprocedural dependency slice.

Promotion applies ordered rules. Candidates with malformed names, helper/test wording, or weak fallback-only evidence are withheld; scenario fixtures are not promoted during an ordinary whole-repository scan. An explicit KPI declaration must have a nearby formula marker, executable calculation, or business metadata. Inferred candidates require the implemented combination of business context and calculation structure; inferred Python symbols additionally require local return-arithmetic evidence. CSV promotion requires explicit KPI and Formula evidence. Rejected candidates are retained in Extraction Review with a reason and suggested action.

Listing 1 summarizes the actual selection path. The extractor deduplicates names within a file and selects one highest-ranked candidate, breaking a rank tie in favor of a human-readable name containing spaces or brackets. A further name filter operates across files. Thus this snapshot does not enumerate every independent calculation in a multi-KPI file, and identical names can collapse distinct implementations. These are implementation boundaries relevant to interpreting dossier counts.

```

for each eligible source file:
  route text to its language/pattern extractor
  construct candidate records with source context
  for each candidate:
    apply name, fixture, and fallback exclusions
    apply explicit-tag or inferred-evidence gates
    retain rejected candidate with review reason
  deduplicate promoted candidates by name
  select the highest-ranked candidate for the file
  assign evidence score from rank and detection kind
  retain selection if its name is not already emitted
return selected records and extraction-review records

```

Listing 1: Source-faithful summary of the frozen candidate-selection procedure.

After promotion, within-file deduplication retains the first candidate with each exact normalized name, before rank selection. Normalization removes supported surrounding delimiters and collapses underscores and whitespace; these final name sets are case-sensitive. Selection maximizes the pair (internal rank, readable-name indicator). If both values

tie, the first candidate in extractor emission order is retained. A cross-file set then keeps the first emitted occurrence of each name. The frozen `os.walk` traversal is unsorted, so duplicate-name retention can depend on traversal order. DAX also performs earlier, route-local name suppression using lowercase names.

Supplement S1, *Frozen method specification and evidence navigation*, provides the complete detection-kind/rank table, route-specific context bounds, predicate locations, emission-order rules, and source hashes. Its machine-readable locator index and verbatim source excerpts accompany the frozen implementation; they specify this snapshot rather than a revised algorithm.

3.4. Heterogeneous artifact coverage

Table 4 separates implemented routes from the distribution of the 38 prepared-record scenarios. The scenario counts describe prepared-record tests, not language-level extraction accuracy. The sample project also supplies files for routes absent from that suite; file presence alone is not an independent validation result. Formula handling depends on supported comments and expression forms, rather than complete compiler semantics for every language family.

3.5. Mathematical annotation and evidence scoring

The Mathematical Annotation and Tagging Engine (MATE) prepares formula evidence for dossier presentation through distinct operations: acquisition of an explicit declaration, extraction of a supported executable expression, normalization for display, and annotation of selected tokens. Its tested behaviors include preserving and normalizing explicit LaTeX, rendering supported expression forms, and linking formula tokens to business and developer perspectives. Formula declarations and executable source are distinct evidence: rendering a declaration does not prove that the implementation computes it. The dossier retains the source context so that this relationship can be inspected and configured conflicts can be surfaced.

A focused application test supplies an explicit yield expression to the LaTeX-normalization path and checks the normalized expression, equation container, numerator/denominator labels, and scale annotation. A separate test inspects the generated Ethylene Production Yield page for mathematical output, the annotated-formula class, and business/developer token attributes. Figure 2 shows these stages. Section 3.8 supplies the concrete source-to-output transformation.

Table 5 makes the operative rules explicit. The parser supports selected DAX forms, SQL aliases, return/assignment expressions, Structured Text, and math-like fallbacks. Before these executable-expression paths, however, it accepts a suitable formula/calculation hint from a comment. At dossier construction, the source file is reloaded when readable; Python uses the function-scoping helper, while other formats retain the available full-file text. This downstream context can differ from the extractor's local candidate window. Supplement S1 specifies the context fallback and ordered branches.

Table 3

Principal candidate-promotion and selection decisions in the frozen extractor. Internal ranks order competing candidates; they are not probabilities.

Candidate route	Priority	Promotion condition	Outcome / boundary
Explicit KPI comment	rank 5	KPI marker plus nearby formula marker, executable calculation, or business meta-data.	Promoted unless the name is invalid/test-like; retains local source context.
Structured explicit forms	rank 3	sql_comment, dax_measure, csharp_attribute, and iec_block are explicit detection kinds; other forms require matching context.	Promoted through the same evidence gate; Full kind mapping is in Supplement S1.
Inferred function/view/alias forms	rank 2	Business context, formula-shaped structure, and executable calculation must co-occur; inferred Python/method routes also require local return arithmetic.	Promoted when all required evidence is present; otherwise routed to Extraction Review.
CSV	route-specific	Explicit KPI and Formula markers and no known bad formula artifact.	Otherwise withheld for review.
Weak fallbacks	rank 1, 0, or -1	plsql_return_fallback, logic-only, forced, and empty-file fallbacks are not sufficient evidence.	Withheld from governed dossier publication.
Generic textual route	rank 2 or 3	Business context plus calculation structure, or explicit metric language plus calculation structure.	May be promoted, subject to the same name and deduplication rules.

Table 4

Implemented routing and available examples in the frozen package. Counts are file inventories, not accuracy or coverage percentages.

Artifact family	Implemented candidate mechanism	Source/formula handling	Available examples
Python	KPI comments; KPI-like function names; local arithmetic promotion gate.	Comment formula and local source window.	22 scenario files; 14 demo files.
SQL	KPI comments; selected aliases, views, and function patterns.	Statement context and supported expressions.	15 scenario files; 4 demo files.
T-SQL / PL/SQL	SQL-like route; PL/SQL function-signature fallback.	SQL-shaped context, not complete dialect semantics.	One .tsql and one .pks demo file; no scenario files.
SAP HANA	SQL-like extraction; JSON view-column expression fallback.	Captured SQL context or constructed expression.	One .hdbview demo file; no scenario files.
ABAP	KPI comments and REPORT naming patterns.	Local context and explicit formula meta-data.	1 scenario file; 2 demo files.
C# / VB.NET	KPI attributes, comments, and selected method names.	Widened context for method/return forms.	2 .cs and 1 .vb demo files; no scenario files.
DAX / Structured Text	DAX measures and comments; Structured Text block/comment patterns.	Supported textual forms and local context.	1 .dax and 1 .st demo file; no scenario files.
Text / CSV	Generic textual candidates subject to promotion gates.	Explicit metadata required for CSV promotion.	1 text demo file; no scenario files.

The nominal *programmatic* formula can therefore be declaration-derived when a qualifying comment remains in the parser input. A formula-like business description is also prepared separately for display selection. The generic mismatch check combines operators inferred from the programmatic display with operators found in filtered source

text. Reused declaration operators can consequently suppress a mismatch signal: these display cues are not invariably independent executable evidence. The Gross Margin Python example below retains its demonstrated mismatch because its scoped executable context lacks the declared subtraction. Normalization removes selected NULLIF, COALESCE, and CAST

Table 5

Formula preparation, conflict signaling, and rendered review-state rules in the frozen artifact. These are structural rules, not semantic-equivalence proofs.

Decision	Implemented rule	Interpretation boundary
Programmatic formula preparation	Accept the first suitable declaration hint in the downstream context before supported executable-expression paths; normalize the selected form for display.	The display can reuse a declaration; it is not invariably independent executable evidence.
Declared formula preparation	Prepare a formula-like business/metadata description separately for mathematical display selection.	Separate processing does not imply an independent origin or executable fidelity.
Generic formula mismatch	Compare declared operator families with the union of programmatic-display cues and filtered source operators; missing families raise a mismatch.	Declaration reuse can mask a conflict. The rule does not establish variable identity, operand order, or semantic equivalence.
Review-state aggregation	Any formula mismatch, governance conflict, audit note, or compliance alarm yields <i>Needs Review</i> ; an approved override can yield <i>Validated</i> ; otherwise a displayed score below 75 yields <i>Needs Review</i> , and the remaining path yields <i>Validated</i> .	Status is an artifact review state, not external certification.

wrappers for presentation, not to preserve all null, conversion, or exceptional-case semantics. Rendered mathematics is not a substitute for inspecting the retained source.

For the Gross Margin example, the declared expression contains subtraction, division, and multiplication, whereas the captured executable return expression contains division and multiplication but no subtraction. The structural operator-family rule therefore contributes a mismatch signal and the rendered status becomes *Needs Review*. The same rule would not, by itself, detect every semantic error that preserves the same operator families; that limitation is why the source expression remains visible for review.

The candidate evidence score is a heuristic, not a probability. In the inspected selection path, base scores for ranks 3, 2, 1, 0, -1 are 95, 85, 75, 60, 50; other ranks default to 70. Designated explicit detection kinds receive a floor of 90, and designated fallback kinds receive a ceiling of 55. This is the implemented rank/kind mapping, not a calibrated weighted sum of tag, formula, structural, and contextual probabilities. Displayed confidence therefore does not establish extraction precision or correctness.

3.6. KPI knowledge record and evidence origins

We define the paper's unit of analysis as a source-linked KPI record

$$K = \langle I, S, F, B, R, P \rangle, \quad (1)$$

where I is identity, S is captured source evidence, F is formula presentation, B is business context, R is review information, and P is the published view. This is a logical description of the implemented record and dossier, not a claim that the software stores this tuple as a dedicated database schema. The artifact carries these fields across extracted dictionaries, prepared details, governance metadata, and generated HTML.

Table 6 distinguishes their origins and rebuild behavior. Some fields originate in source comments or configured defaults, not an independent business authority. A displayed owner therefore records an assignment in the artifact; it does not by itself prove organizational approval. The software manifest identifies the evaluated source revision at package level. We do not claim that every displayed field contains its own persisted revision identifier or that display identifiers are stable under all source renames.

3.7. Review signals and bounded governance

The reviewer can inspect a dossier, locate its source, and distinguish formula conflicts, technical markers, ownership conditions, and rule-mapped alarms. The interface can prepare review text and an email draft; it does not persist an acknowledged or resolved workflow state. The artifact does not establish general equivalence between arbitrary code and natural-language requirements.

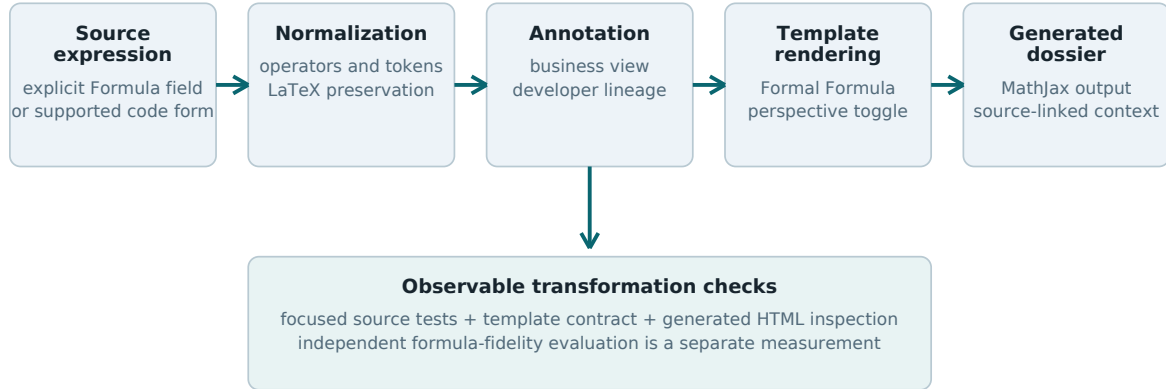
The application tests exercise five configured rule mappings associated with Saudi PDPL, NCA ECC, GDPR, and the AICPA Trust Services Criteria (Saudi Data & AI Authority, 2026; National Cybersecurity Authority, 2025; European Union, 2016; AICPA, 2023). Two additional test functions within the same application-test count exercise two bounded ISO 22400 templates using three conflict inputs and two clean input examples (International Organization for Standardization, 2014). These verify selected review behaviors, not exhaustive control coverage or compliance certification. Broader governance effectiveness is outside this paper's scope.

3.8. Worked example: Ethylene Production Yield

The archived demonstration file `control_no_conflict_yield.py` and its generated `ethylene_production_yield.html` dossier provide an inspectable end-to-end example. This is a curated demonstration already in the frozen artifact, not a

MATE formula annotation pipeline

Supported source and declared expressions remain linked to their presentation.



MATE: Mathematical Annotation and Tagging Engine

Figure 2: MATE prepares supported formula evidence for mathematical and token-level presentation. Tests check specified transformations and output attributes; independent formula fidelity is a separate evaluation.

Table 6

Logical KPI record: evidence origins and update boundaries in the evaluated artifact.

Component	Evidence or representation	Origin / responsibility	Rebuild boundary
Identity (I)	Name, detection kind, displayed KPI identifier.	Normalized source declaration or inferred candidate; generated presentation identity.	Recomputed or matched by the implemented name rules; not universal entity resolution.
Source (S)	File path, line anchor, language, code context.	Captured by extraction; package manifest identifies the frozen software.	Re-read for the selected source scope.
Formula (F)	Declared/supported expression, formal formula, token annotations.	Source metadata or supported calculation pattern; MATE prepares presentation.	Reconstructed for the dossier; fidelity requires separate checking.
Business context (B)	Description, objective, unit, input and reporting-source fields.	Explicit source/KB metadata or template-derived text, with optional authoring outside this evaluation.	Prepared from current available inputs; no general persistence guarantee is asserted.
Review (R)	Owner assignment, conflicts, technical notes, rule metadata, status.	Configured metadata and rule evaluation; human interpretation remains distinct.	Recomputed signals; communication controls do not persist resolution.
Publication (P)	Dossier, formula perspectives, source/context and portfolio links.	Templates and generated HTML over the prepared record.	Regenerated output, not an independent evidentiary source.

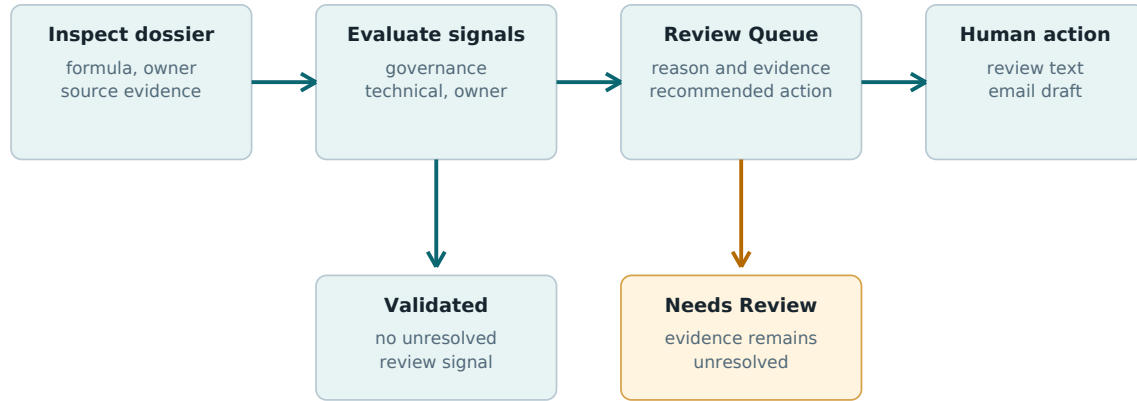
new industrial accuracy experiment. Listing 2 reproduces the relevant declaration and calculation; other metadata comments are omitted for space.

Candidate and provenance. The explicit comment provides the display name; neighboring formula and business metadata satisfy the promotion path. The archived dossier

records the source filename, line anchor 1, Python language, and comment-based definition pattern. This anchor refers to the declaration, while the calculation remains visible in the captured source context.

Evidence-driven review workflow

Distinct signals remain separate so reviewers can act on the actual reason for review.



Communication actions support review; they do not persist an acknowledged or resolved state.

Review-state description aligned with frozen software snapshot

Figure 3: Evidence-driven review: distinct signals retain their reasons and evidence. Communication actions support review without constituting a persisted resolution state.

```

1 # KPI: Ethylene Production Yield
2 # Formula: Ethylene Yield % = (Ethylene Produced (tons) / Feedstock Input (tons)
3   ) * 100
4 # Unit: %
5 # Accountable Owner: Operations Data Owner
6
7 def calculate_ethylene_yield_pct(ethylene_produced_tons: float,
8   feedstock_input_tons: float) -> float:
9   if feedstock_input_tons <= 0:
10    return 0.0
11   return (ethylene_produced_tons / feedstock_input_tons) * 100.0
  
```

Listing 2: Selected lines from the archived Ethylene Production Yield example. Other metadata comments are omitted.

Mathematical presentation. The displayed equation is

$$\text{Ethylene Production Yield} = \frac{\text{Ethylene Produced}}{\text{Feedstock Input}} \times 100. \quad (2)$$

The published labels identify the numerator and denominator in tons and the scale as $\times 100$. The annotated view binds the business token *Ethylene Produced* to `VARIABLE: ethylene_produced_tons` and exposes `OPERATOR: /` in the developer perspective. These strings are present in the archived HTML and checked by the focused MATE test. The non-positive-input guard remains visible in the source; the displayed ratio alone is not a complete encoding of exceptional behavior.

Business context and review. The source declares the description, objective, input units, reporting source, usage,

and accountable owner. The generated page carries these fields and displays *Validated* with *Operations Data Owner*. Here *Validated* is the artifact’s status for the configured checks, not an independent certification. Optional AI-assisted authoring was not needed to generate this record.

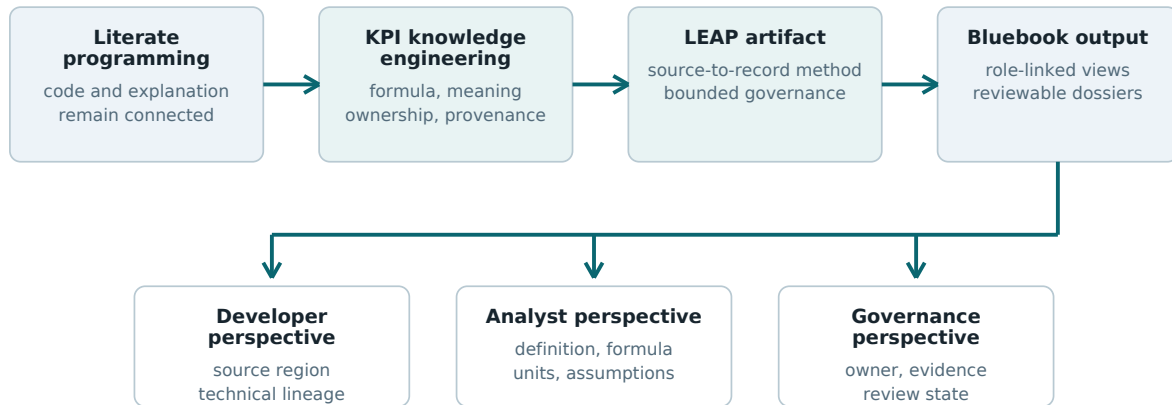
Contrast with a conflict example. The same sample project includes Gross Margin Percentage: its declared formula is $(\text{Revenue} - \text{Cost}) / \text{Revenue} \times 100$, while the source return expression computes $(\text{Revenue} / \text{Cost}) \times 100$. Because the declared expression contains subtraction while the executable evidence does not, the implemented operator-family check contributes a mismatch signal and the archived dossier displays *Needs Review*. This illustrates why rendered mathematics and executable evidence must remain distinguishable; it does not establish a general semantic-equivalence detector. The interface captures in Figures 5 and 6 show the corresponding presentation concepts.

3.9. Interactive Bluebook and actual interface

The *Interactive Bluebook* presents linked perspectives over the record: developers inspect source and tokens; analysts inspect meaning, formula, and units; reviewers inspect ownership and signals. Figure 4 relates these views to the literate-programming foundation. The boundary-object interpretation is a design rationale (Star and Griesemer, 1989); this technical paper does not test collaborative outcomes.

From literate programming to governed KPI knowledge

The Interactive Bluebook is a publication surface over a shared evidence record.



One knowledge record, multiple evidence-preserving perspectives

Figure 4: Literate programming motivates a shared KPI record whose published views retain the connection between implementation and explanation.

Table 7

Technical evidence streams and their interpretation.

Evidence stream	Scope	Observable or demonstrated behavior	RQ
Architecture and worked example	Frozen source and archived generated dossier	Source, declared context, mathematical tokens, and review state linked within a record.	RQ1
Application verification	18 functions	Routes/assets, focused candidate promotion, MATE assertions, and bounded rule cases.	RQ2
Prepared-record scenarios	38 curated fixtures	Details, review status, expected signal text, and rule/alarm metadata for prepared inputs.	RQ2
Generation and page checks	28 dossiers; 32 HTML pages; 6 routes	Successful publication, page-validation output, and route availability.	RQ2
RR-004, summarized by RR-005	Four archived 62-record runs	Selected input counts, output counts, and wall time for one curated scope.	RQ3
RR-006	One broad repository run	Inventory, selected implementation scope, output count, and wall time.	RQ3

4. Evaluation Method

4.1. Evidence design and artifact identity

The evaluation separates architectural demonstration, repository discovery, prepared-record behavior, mathematical presentation, and archived runtime observations. Table 7 states which evidence answers each research question. A passing test supports only the behaviors it invokes and the assertions it checks; the archive does not contain an independently adjudicated KPI or formula gold set.

The frozen research-artifact snapshot derives from commit f3b7b1d7af61; the full identity and journal-specific

changes are recorded in software/SOFTWARE_SNAPSHOT.md. The SHA-256 manifest identifies the evaluated files. QA-003 records the frozen-snapshot application tests, scenario checks, sample-project build, generated-page validation, and route checks. A separate historical-fixture audit traces the 38-fixture suite to commit 9c1f1d81d989 and confirms that all 38 fixture blobs are byte-identical at the frozen research baseline. Earlier QA states remain separate rather than being added to the test count.

All reported verification uses optional AI-assisted authoring disabled. Survey aggregates, meeting observations, and AI-quality data are not evidence streams for this paper. The

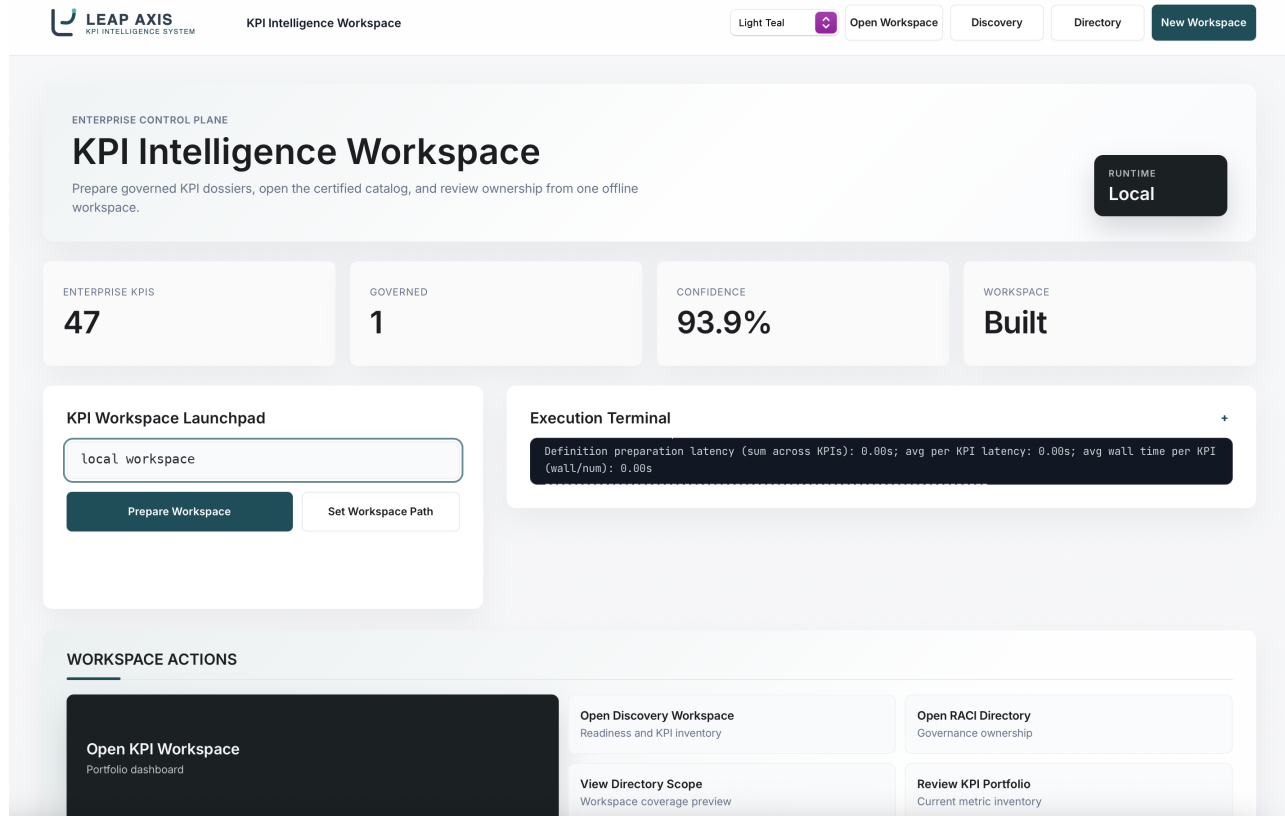
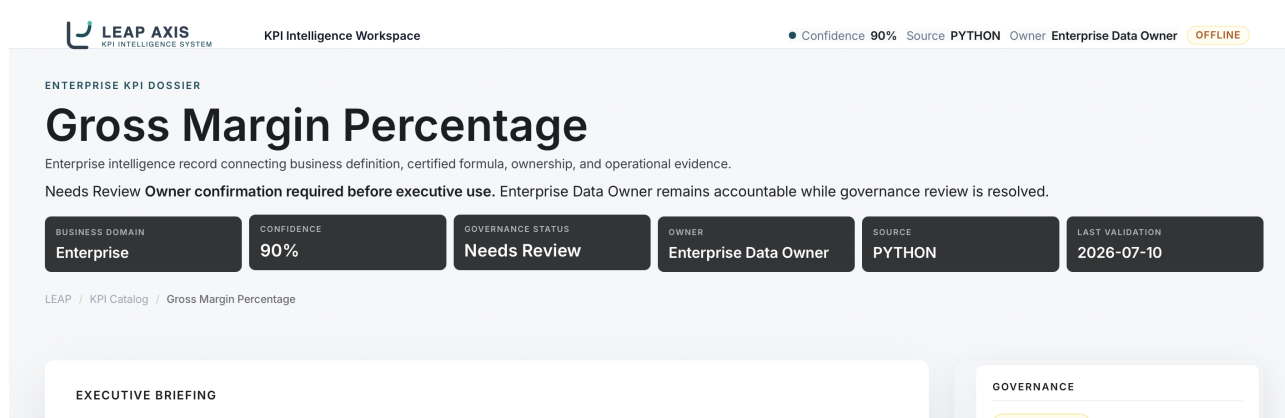
(a) KPI Intelligence Workspace — dashboard**(b) Gross Margin Percentage — dossier and review state**

Figure 5: Actual LEAP interface captures supplied with the research archive: (a) the KPI Intelligence Workspace dashboard; (b) the Gross Margin Percentage dossier header and review state. Displayed counts and confidence scores represent interface state and are not extraction-accuracy measurements; interface terms such as “certified” or “governed” do not by themselves establish external certification.

architectural demonstration does not assert a formal proof or independent user validation of the design.

4.2. Application and generated-output checks

Application tests invoke different layers. Route and static-asset tests check the control plane and served pages. Candidate-promotion tests call the repository-discovery entry point on temporary positive or excluded examples.

MATE tests assert specified normalized LaTeX and selected generated-HTML attributes. Rule tests evaluate configured conflict inputs and clean controls. The frozen sources contain 18 application-test functions and one suite-level test of the 38 scenarios. These account for the 19 checks in the archived pytest report. The two ISO-template functions are included within the application-test layer, not an additional

(a) Formula and linked perspectives

MATHEMATICAL FORMULATION

FORMAL FORMULA

$$\text{Gross Margin Percentage} = \frac{(\text{Revenue} - \text{Cost})}{\text{Revenue}} \times 100$$

Where:

- *Numerator*: Revenue - Cost
- *Denominator*: Revenue
- *Scale*: $\times 100$

ANNOTATED FORMULA

Business Interpretation Developer Lineage

Total Sales Revenue	Minus	Cost	Divided By	Total Sales Revenue	Scaled by 100
Revenue	-	Cost	÷	Revenue	$\times 100$

(b) Source provenance

EVIDENCE & LINEAGE

Operational Logic Formula Lineage Provenance Inputs

BUSINESS DATA SYSTEM	EXPLICIT SOURCE: Finance ERP (SAP FI-CO module)	SOURCE FILE	conflict_formula_margin.py
SOURCE LINE	1	DEFINITION PATTERN	Comment
GOVERNANCE DOMAIN	Enterprise	KPI ID	KPI-440AE64A44

Figure 6: Details from the Gross Margin Percentage dossier: (a) declared formal formula and linked business/developer token perspectives; (b) provenance fields linking the dossier to `conflict_formula_margin.py`. The annotated strip is a token-oriented perspective and does not encode operator precedence; the formal formula above it is the grouping-preserving mathematical expression. The formula remains visible when the dossier requires review, preserving the distinction between declared mathematical meaning and executable evidence.

independent study; the suite-level check is not an additional scenario.

The archived generation check produced 28 dossiers and 32 HTML pages. Generated-HTML validation and six representative route checks are separate checks of publication and availability. Neither an HTTP success code nor a warning-free

documentation build demonstrates the semantic correctness of every displayed statement.

4.3. Controlled prepared-record scenarios

The 38 scenario files are curated, source-shaped examples with embedded expected-result headers: 22 Python, 15 SQL,

Table 8

Mapping from major method branches to existing focused application checks. Function names identify the executable checks in the frozen package; counts are not accuracy observations.

Method branch	Representative frozen test functions	What is asserted
Repository discovery / promotion	<code>test_candidate_promotion_gate_quarantines_helper_and_register_noise</code> ; <code>test_candidate_promotion_gate_promotes_explicit_formula_kpi</code>	Bounded negative and positive promotion behavior on temporary repository inputs.
MATE normalization / publication	<code>test_mate_preserves_explicit_latex_and_formula_annotations</code> ; <code>test_mate_generated_dossier_contains_annotated_formula_lineage</code>	Selected LaTeX normalization, annotation attributes, and generated-page mathematical lineage.
Rule routing / review signals	Five compliance-alarm tests plus <code>test_review_queue_signal_separation</code>	Rule identity/metadata for configured cases and separation of review reasons.
ISO template checks	<code>test_iso_22400_yield_rule_detection_and_clean_control</code> ; <code>test_iso_22400_asset_utilization_rule_detection_and_clean_control</code>	Three conflict inputs and two clean inputs across two narrow templates.
Publication / availability	Route, committed-page, static-asset, and workspace-asset tests	Page/asset availability and control-plane response, not semantic correctness.

and one ABAP fixture. Table 9 groups their numbered files by intended test purpose. These groups are a transparent inventory of the fixture suite, not classes sampled from an enterprise population. The expectations are treated as development-time acceptance criteria; the archive does not establish independent authorship or adjudication of their text.

An audit of archived Git provenance first locates the fixtures, their expected-result headers, and the harness together at commit `9c1f1d81d989` on 15 July 2026. All 38 fixture files remain byte-identical at research baseline `f3b7b1d7af61`. A processed inventory dated 14 July reports 38/38 passing, but no contemporaneous raw execution transcript, environment receipt, or separately versioned pre-execution oracle was located. The suite is therefore reused as historical prepared-record acceptance evidence, not represented as a newly designed or independently adjudicated experiment. The accompanying historical-fixture audit records the full commit identities, comparison results, and source paths. The supplied QA-004 execution summary reports that `python data/validation/test_scenario_fixtures.py` completed on 21 September 2026 with exit status 0 and 38/38 passing. This review distribution preserves that summary as a reported-run record, not as the original stdout, stderr, or environment receipt. The available QA-003 archive and the historical inventory remain the packaged verification sources; no later rerun authenticates the historical execution date or supplies an independent oracle.

For each fixture, the harness reads the expected header, parses its KPI name, and constructs a record containing the filename, full fixture context, language hint, and a fixed input confidence of 90.0. It calls `generate_kpi_details` and `attach_governance`. It then checks derived status and Review Queue membership, expected rule identifiers and metadata, non-empty alarm evidence and recommended action, clean-control absence of alarms, and specified signal text. A fixture passes when this assertion set reports no mismatch.

Table 9

Scenario inventory by file-number group. Counts sum to 38; groups describe fixture intent, not independent measurement strata.

Files	Intended scenario group	Count
01–03	Clean Python, SQL, and ABAP examples	3
04–08	Formula/source conflict or absent-evidence cases	5
09–12	Technical markers and suspicious fallback	4
13–17	Rule-mapped positive cases	5
18–22	Corresponding clean rule controls	5
23–28	Combined signals, review/owner conditions, clean status	6
29–38	Formula-shaped examples and presentation inputs	10
Total		38

This harness does not call the repository-discovery entry point or candidate-promotion gate. Its 38 passing results therefore concern downstream behavior on prepared records; repository discovery is covered separately by application tests and archived generation records. Although the header parser reads an expected-formula field, the harness does not directly compare that field with a generated formula. Exact normalization and selected annotation attributes are checked by the separate MATE application tests. The ten formula-shaped scenarios must not be counted as ten independent formula-equality validations.

For example, the restricted-execution fixture expects an NCA-linked signal, while its clean counterpart expects no such alarm. Formula-conflict and TODO examples check whether the configured review reasons are surfaced. These are demonstrations of configured behavioral conformance, not estimates of general detection sensitivity or false-positive rate.

4.4. Archived runtime observations

RR-004 preserves four executions of a curated workload with 65 selected files and 527 analyzed lines. RR-005 is their derived repeatability summary, not another set of runs. RR-006 records one broad local-repository traversal with a different inventory and curation scope. We report the counts and instrumented wall times associated with each archive and do not combine the two scopes into a scaling experiment.

The measures are output-record count, selected source size, and total wall time. For the four repeated runs, we report the arithmetic mean and sample standard deviation. Complete hardware, warm-up, and cache metadata are not archived, so comparisons to other tools or environments are outside the evidence. Internal confidence scores are not used as accuracy statistics.

5. Results

5.1. RQ1: Source-linked KPI knowledge construction

The architecture operationalizes literate coupling through the record components in Table 6 and the implemented selection procedure in Listing 1. The Ethylene Production Yield example links a declared KPI, local calculation, annotated expression, source anchor, business metadata, and a generated dossier. The contrasting Gross Margin example shows why a declared mathematical view and a conflicting return expression remain separately inspectable. Together, these demonstrate the record-construction mechanism in the frozen artifact; they do not establish how frequently that mechanism succeeds on an independently labeled corpus.

5.2. RQ2: Behavior demonstrated by the verification layers

QA-003 reports 19 passing regression checks and a passing execution of the 38 prepared-record scenarios. The supplied sources define 18 application-test functions and one suite-level scenario test; these are distinct counting units from the 38 fixture inputs. The archived July inventory and QA-003 involve the same fixture suite; their scenario counts are not added. The sample-project build produced 28 dossiers and 32 HTML pages; generated-page validation reported zero warnings and errors, and six representative routes returned HTTP 200. Table 10 retains the counting units and separates test functions, scenarios, and generated outputs.

The focused MATE assertions confirm the specified normalization and selected business/developer annotations. The scenario suite confirms expected status, signals, and rule metadata for its prepared inputs. Two application-test functions exercise two ISO templates using three conflict inputs and two clean input examples. These results support the behavior of those layers and their configured cases, not exhaustive language, formula, or standards coverage.

5.3. RQ3: Execution observations at two input scopes

Four archived runs with AI-assisted authoring disabled used the same curated input scope: 65 selected source files,

Table 10

Archived frozen-artifact verification; layers are not additive estimates of extraction accuracy.

Verification layer	Count	Outcome
Regression checks	19	Passed: 18 application functions and one suite-level check
Prepared-record scenarios	38	Historical inventory and QA-003 report passing
Generated KPI dossiers	28	Build completed
Generated HTML pages	32	No warnings/errors
Representative routes	6	HTTP 200

Table 11

Repeated executions in RR-004; RR-005 summarizes these same four runs.

Run	Selected files	Lines	Records	Wall time (s)
RR-004-A	65	527	62	0.93
RR-004-B	65	527	62	1.14
RR-004-C	65	527	62	0.95
RR-004-D	65	527	62	1.16

Table 12

RR-006 broader repository execution observation.

Observed quantity	Value
Workspace items	4,488
Files / directories	4,084 / 404
Eligible files	454
Implementation files / lines	195 / 4,539
Generated dossiers	148
Wall time	2.73 s

527 analyzed lines, and 62 generated KPI records. Their wall times were 0.93, 1.14, 0.95, and 1.16 seconds (Table 11). The minimum was 0.93 seconds, the maximum 1.16 seconds, the arithmetic mean 1.045 seconds, and the sample standard deviation 0.122 seconds. Hardware, cache, and warm-up metadata are incomplete, so these values describe the archived environment and are not presented as a portable benchmark.

A separate broad local-repository run inventoried 4,488 workspace items, including 4,084 files and 404 directories. It retained 454 eligible files, evaluated 195 implementation files and 4,539 implementation lines, and generated 148 dossiers in 2.73 seconds (Table 12). This run intentionally exercised a heterogeneous repository and therefore includes scan noise that would not belong to a curated production KPI corpus. It is reported as a broader repository execution observation, not as an accuracy result, controlled stress test, or scaling law, and is not normalized against the curated run.

Both runtime records include internal evidence scores used for review routing. Because no calibration study is archived, those scores are not interpreted as correctness probabilities or accuracy percentages.

6. Discussion

6.1. Literate reconstruction and evidence boundaries

The principal contribution is the inspectable connection among executable evidence, declared meaning, mathematical presentation, and a published KPI record. Unlike a detached code summary, the record exposes the source and review information needed to question its interpretation. The different verification layers demonstrate specified parts of that construction. Their successful execution does not establish corpus-level extraction accuracy.

Optional AI-assisted authoring remains an extension over an already constructed record. Separating authoring from source analysis and status assignment makes the origin and responsibility of each output clearer. The fixed rules make the configured procedure inspectable, subject to the documented order-dependent selection. A repeatable implementation can still contain a systematic error, so repeatability and correctness remain separate properties.

6.2. Interpreting the runtime observations

The four curated runs demonstrate repeatability of the observed output count and provide a bounded wall-time interval for one archived workload. RR-006 shows that the same implementation can complete a substantially broader local-repository traversal and dossier build. The two records differ in scope, curation, and repository composition; a per-KPI comparison or extrapolated scaling claim would therefore be misleading. A formal performance study requires fixed hardware, cache and warm-up controls, repeated broad runs, and predefined workload strata.

6.3. Mathematical and governance traceability

MATE supplies an inspectable representation of formula structure and connects mathematical output to source evidence. The current evidence supports implementation traceability, not a formula-fidelity percentage. Likewise, the governance layer demonstrates specified behavior for five configured rule mappings and two narrow ISO 22400 templates. The contribution is the explicit route from rule metadata and evidence to a review signal, not exhaustive coverage of the named standards.

6.4. The Bluebook as a boundary object

The Bluebook presents linked technical, analytical, and governance views over the same recovered record. This makes disagreements more locatable: a reviewer can point to a formula, unit, source span, owner, or evidence gap rather than reconcile disconnected documents. Whether this shared representation improves cross-functional collaboration is intentionally left to a separate human-centered study rather than inferred in this technical paper.

6.5. Implications for software engineering practice

The supported evidence suggests three implementation principles:

1. Preserve the distinction between executable evidence, declared metadata, and authored explanation.
2. Treat generated explanations, when enabled, as derived and reviewable rather than evidentiary sources.
3. Preserve provenance at the granularity the artifact can support: source anchors for captured code evidence, token links for selected mathematical elements, and explicit origin categories for curated, template-derived, or optionally authored fields.

6.6. Limitations and threats to validity

Extraction and formula validity. No completed adjudicated KPI gold set or formula gold set is archived. The paper therefore does not report precision, recall, false-positive rate, formula accuracy, or ablation effects. Controlled fixtures verify expected software behavior but cannot replace independent annotation.

Runtime validity. The curated runtime records lack complete hardware, cache, and warm-up metadata. RR-006 is a single broader repository execution observation and includes repository noise. The timings are reproducibility records for their archived inputs, not general performance guarantees.

External validity. The tested languages, source patterns, and governance rules are bounded by the frozen fixtures and configured mappings. The current selection path retains one candidate per file and filters duplicate names across files; this can omit independent metrics or merge names that refer to different calculations. Captured line anchors and context windows are not whole-program dependency slices. Highly dynamic code, additional dialects, and organization-specific vocabularies require separate validation.

Oracle and presentation validity. Scenario expectations are development artifacts rather than an independently adjudicated oracle. The prepared-record harness does not test repository discovery or directly assert its parsed expected-formula field. Some dossier text and ownership assignments come from templates or defaults; those fields must not be mistaken for independently approved business facts. The interface figures are illustrative captures and are not themselves counted as software-verification outcomes.

Provenance granularity. The artifact preserves record-level source anchors, captured context, selected mathematical token links, and field-family origin categories. It does not persist an independent revision identifier or machine-verifiable provenance edge for every displayed value or sentence. Claims in this paper therefore refer to source-linked records and supported field origins, not universal per-value provenance.

Measurement interpretation. Internal evidence scores rank recovered records for presentation and review routing. They have not been calibrated as probabilities and must not be interpreted as extraction accuracy or confidence intervals.

7. Conclusion

We presented LEAP, a literate-programming architecture for reconstructing traceable KPI knowledge from enterprise source artifacts. The contribution is a method and record structure that keep source context, mathematical presentation, business metadata, and review signals connected through an Interactive Bluebook. The worked example and implementation-faithful specification make this transformation inspectable rather than treating generated documentation as an independent source of truth.

The archived verification reports 19 regression checks passing, and the historical archive reports all 38 prepared-record scenarios passing. The packaged QA-003 record also reports the unchanged 38-fixture suite passing; these records are not combined as independent observations. Four archived runs each generated 62 records from the same 65 selected files in 0.93–1.16 seconds; a broader run generated 148 dossiers in 2.73 seconds. These results establish bounded implementation behavior and execution observations. They do not substitute for independent extraction or formula-fidelity measurement.

Future technical evaluation will address adjudicated KPI and formula corpora, multi-KPI-per-file selection, and controlled runtime workloads. Broader governance effectiveness, AI-assisted authoring quality, and cross-functional organizational effects remain separate studies. This paper's focus is the source-to-knowledge architecture and the explicit connection between implementation and explanation.

Data and Artifact Availability

The accompanying review workspace supplies the frozen software, tests, prepared-record fixtures, generated outputs, evidence manifests, manuscript source, historical-fixture audit, and QA-003 verification records. Supplement S1 identifies the exact method branches, source hashes, worked-example locations, and evidence paths. A separate manuscript-source archive contains the build materials and supplement, not the complete executable artifact; reviewers should receive the review workspace as well as the PDF.

QA-004 is represented here by the supplied execution summary. Its original run directory has not been transferred into this distribution and is not claimed to be included. The package's evidence-status record makes that distinction explicit. No public archival identifier is asserted.

Funding

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used OpenAI ChatGPT and Codex, and Google Gemini, to assist with language refinement, manuscript organization, consistency checking, literature-navigation support, and reproducibility-package organization. The authors independently reviewed, verified, and edited the resulting material and take full responsibility for the content of the article.

References

- E. Aghajani, C. Nagy, O.L. Vega-Márquez, M. Linares-Vásquez, L. Moreno, G. Bavota, M. Lanza, Software documentation issues unveiled, in: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), 2019, pp. 1199–1210. <https://doi.org/10.1109/ICSE.2019.00122>.
- E. Alor, S. Khatoonabadi, E. Shihab, Evaluating the use of LLMs for documentation to code traceability, *ACM Transactions on Software Engineering and Methodology* (2026). <https://doi.org/10.1145/3819820>.
- D. Fucci, E. Alégroth, T. Axelsson, When traceability goes awry: An industrial experience report, *Journal of Systems and Software* 192 (2022) 111389. <https://doi.org/10.1016/j.jss.2022.111389>.
- X. Gao, Executable Code Knowledge: Code as a native, validation-carrying knowledge representation for AI coding agents, *arXiv preprint arXiv:2608.16295* (2026). <https://arxiv.org/abs/2608.16295>.
- International Organization for Standardization, ISO 22400-2:2014, Automation systems and integration—Key performance indicators (KPIs) for manufacturing operations management—Part 2: Definitions and descriptions, 2014. <https://www.iso.org/standard/54497.html>.
- P. Mäder, O. Gotel, Towards automated traceability maintenance, *Journal of Systems and Software* 85 (10) (2012) 2205–2227. <https://doi.org/10.1016/j.jss.2011.10.023>.
- National Cybersecurity Authority, Essential Cybersecurity Controls (ECC 2-2024), Saudi Arabia, updated 2025. <https://nca.gov.sa/en/regulatory-documents/controls-list/ecc/>.
- J.F. Pimentel, L. Murta, V. Braganholo, J. Freire, A large-scale study about quality and reproducibility of Jupyter notebooks, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), 2019, pp. 507–517. <https://doi.org/10.1109/MSR.2019.00077>.
- Saudi Data & AI Authority, Personal Data Protection Law and implementing regulatory resources, Saudi Arabia, accessed September 2026. <https://sdaia.gov.sa/en/SDAIA/about/Pages/RegulationsAndPolicies.aspx>.
- European Union, Regulation (EU) 2016/679 (General Data Protection Regulation), Official Journal of the European Union, 2016. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- AICPA & CIMA, 2017 Trust Services Criteria (with revised points of focus—2022), resource release, 2023. [AICPA Trust Services Criteria resource](https://aicpa.org/2023/07/20/2023-trust-services-criteria/).
- L. Wang, C. Sun, C. Zhang, W. Nie, K. Huang, Application of knowledge graph in software engineering field: A systematic literature review, *Information and Software Technology* 164 (2023) 107327. <https://doi.org/10.1016/j.infsof.2023.107327>.
- Alation, Alation documentation: Data catalog, discovery, governance, and lineage, Product documentation, accessed July 14, 2026. <https://docs.alation.com/>.
- M. Allamanis, E.T. Barr, P. Devanbu, C. Sutton, A survey of machine learning for big code and naturalness, *ACM Computing Surveys* 51 (4) (2018) 81:1–81:37. <https://doi.org/10.1145/3212695>.
- Collibra, Collibra product documentation: Data catalog, governance, stewardship, and lineage, Product documentation, accessed July 14, 2026. <https://productresources.collibra.com/docs/collibra/latest/>.
- W.W. Eckerson, Performance Dashboards: Measuring, Monitoring, and Managing Your Business, John Wiley & Sons, 2011.
- R.S. Kaplan, D.P. Norton, The Balanced Scorecard: Translating Strategy into Action, Harvard Business School Press, 1996.

- D.E. Knuth, Literate programming, *The Computer Journal* 27 (2) (1984) 97–111. <https://doi.org/10.1093/comjnl/27.2.97>.
- SAP, SAP LeanIX documentation: Enterprise architecture management and application portfolio capabilities, Product documentation, accessed July 14, 2026. <https://help.sap.com/docs/leanix>.
- B. Marr, *Key Performance Indicators: The 75 Measures Every Manager Needs to Know*, Pearson, 2012.
- Palantir Technologies, Foundry documentation: Ontology, data integration, lineage, and code workspaces, Product documentation, accessed July 14, 2026. <https://www.palantir.com/docs/foundry/>.
- D. Parmenter, *Key Performance Indicators: Developing, Implementing, and Using Winning KPIs*, John Wiley & Sons, 2015.
- T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, P. Ivanov, D. Avila, S. Abdalla, C. Willing, Jupyter Development Team, Jupyter Notebooks—a publishing format for reproducible computational workflows, in: *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, IOS Press, 2016, pp. 87–90. <https://doi.org/10.3233/978-1-61499-649-1-87>.
- A. Rule, A. Tabard, J.D. Hollan, Exploration and explanation in computational notebooks, in: *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018, pp. 1–12. <https://doi.org/10.1145/3173574.3173606>.
- Sourcegraph, Sourcegraph documentation: Code search, code navigation, and code intelligence, Product documentation, accessed July 14, 2026. <https://sourcegraph.com/docs>.
- R. van Solingen, V. Basili, G. Caldiera, H.D. Rombach, Goal Question Metric (GQM) approach, in: *Encyclopedia of Software Engineering*, Wiley, 2002.
- E. Domínguez, B. Pérez, A.L. Rubio, M.A. Zapata, A flexible framework to ensure traceability, consistency, and propagation of KPIs evolution, *Journal of Software: Evolution and Process* 37 (2) (2025) e70004. <https://doi.org/10.1002/smr.70004>.
- E. Putrycz, A.W. Kark, Connecting legacy code, business rules and documentation, in: *Rule Representation, Interchange and Reasoning on the Web (RuleML 2008)*, Lecture Notes in Computer Science 5321, Springer, 2008, pp. 17–30. https://doi.org/10.1007/978-3-540-88808-6_5.
- J. Carette, S.W. Smith, J. Balaci, Generating software for well-understood domains, in: *Eelco Visser Commemorative Symposium (EVCS 2023)*, OASIs 109, 2023, pp. 7:1–7:12. <https://doi.org/10.4230/OASIs.EVCS.2023.7>.
- K. Nørmark, Elucidative programming, *Nordic Journal of Computing* 7 (2) (2000) 87–105. <https://homes.cs.aau.dk/~nørmark/elucidative-programming/papers/njc-paper.pdf>.
- A. Kelious, C. Tahri, E. Bardet, CODENS: Transforming code changes into living, accessible, and queryable documentation, in: *Proceedings of the 2026 ACM Symposium on Document Engineering (DocEng '26)*, ACM, 2026, 4 pages. <https://doi.org/10.1145/3820755.3832807>.
- M. De Sanctis, L. Iovino, M.T. Rossi, M. Wimmer, MIKADO: a smart city KPIs assessment modeling framework, *Software and Systems Modeling* 21 (2022) 281–309. <https://doi.org/10.1007/s10270-021-00907-9>.
- A. Marcus, J.I. Maletic, A. Sergeyev, Recovery of traceability links between software documentation and source code, *International Journal of Software Engineering and Knowledge Engineering* 15 (5) (2005) 811–836. <https://doi.org/10.1142/S0218194005002543>.
- S.L. Star, J.R. Griesemer, Institutional ecology, 'translations' and boundary objects: Amateurs and professionals in Berkeley's Museum of Vertebrate Zoology, 1907–39, *Social Studies of Science* 19 (3) (1989) 387–420. [doi:10.1177/030631289019003001](https://doi.org/10.1177/030631289019003001).
- M. Moser, J. Pichler, G. Fleck, M. Witlatschil, RbG: A documentation generator for scientific and engineering software, in: *2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, IEEE, 2015, pp. 464–468. <https://doi.org/10.1109/SANER.2015.7081857>.
- O. Moseler, F. Lemmer, S. Baltes, S. Diehl, On the diversity and frequency of code related to mathematical formulas in real-world Java projects, *Journal of Systems and Software* 172 (2021) 110863. <https://doi.org/10.1016/j.jss.2020.110863>.
- M. De Sanctis, L. Iovino, M.T. Rossi, Ethical risk traceability in smart city KPIs assessment, *IET Smart Cities* 7 (2025) e70012. <https://doi.org/10.1049/smc2.70012>.